

BANCO DAVIVIENDA SALVADOREÑO · BANCA EMPRESAS

Manual técnico

Instalación y configuración de la API y el frontend

Financiamiento de Cuentas por Pagar

Financiamiento de Cuentas por Pagar. Este documento explica cómo instalar y arrancar los dos proyectos, y cómo se conectan. El uso de las pantallas está en el Manual de usuario (carpeta 03). El detalle de migraciones y del script de producción está en `CXP_DAV_SERVER_REFACTOR/db/README.md`.

Pieza	Carpeta	Stack
API	<code>CXP_DAV_SERVER_REFACTOR</code>	Java 21, Spring Boot 3.3.0, PostgreSQL 16. Empaquetado <code>war</code> . Artefacto <code>APIFinanciamientoEmpresas</code> .
Frontend	<code>CXP_DAV_CLIENT_REFACTOR</code>	React 19, Vite 6, Tailwind 4. Node.js 18 o superior.

La API escucha en el puerto **8080**. El front de desarrollo escucha en el **5173** y se publica bajo la ruta `/financiamientocuentasporpagar/`.

1. Desarrollo local

Hacen falta Java 21, Maven, Node.js 18 o superior y PostgreSQL 16. El cliente debe poder llamar a la API desde el navegador; el origen por defecto permitido es `http://localhost:5173`.

1.1 Base de datos

Crear una base vacía. Si no se definen variables, la API usa `jdbc:postgresql://localhost:5432/postgres` con usuario y contraseña `postgres`.

Al primer arranque, Flyway aplica `src/main/resources/db/migration` (V1 a V15): tablas, catálogos, rutas, menús y parámetros. Hibernate solo valida el esquema (`ddl-auto=validate`).

Datos de demostración, solo en desarrollo, después de ese primer arranque. Ejecutar desde `CXP_DAV_SERVER_REFACTOR`:

```
psql -h localhost -U postgres -d postgres -f db/dev/seed_dev.sql
```

El acceso de desarrollo es `POST /api/v1/auth/sso-login` con el DUI en el cuerpo. Usuarios del seed:

DUI	Rol
<code>00000000-0</code>	<code>ADMIN</code> (operador bancario)
<code>11111111-1</code>	<code>PAYER</code>
<code>22222222-2</code>	<code>SUPPLIER</code>

DUI	Rol
44444444-4	SYSTEM_ADMIN (parámetros)

1.2 API

Copiar `CXP_DAV_SERVER_REFACTOR/.env.example` a `.env` solo si la base no es la local por defecto. El IDE carga ese archivo con `.vscode/launch.json` (configuración `FactorajeApplication`). También se importa desde `application.properties`, y una variable de entorno real tiene prioridad sobre el archivo.

Variable	Default	Uso
DB_URL	<code>jdbc:postgresql://localhost:5432/postgres</code>	JDBC de PostgreSQL
DB_USERNAME	<code>postgres</code>	Usuario
DB_PASSWORD	<code>postgres</code>	Contraseña
FLYWAY_ENABLED	<code>true</code>	<code>false</code> en producción, donde el esquema lo carga el DBA

Desde `CXP_DAV_SERVER_REFACTOR`:

```
mvn spring-boot:run
```

La clase principal es `com.davivienda.factoraje.FactorajeApplication`. El log de archivo queda en `./logs/APIFinanciamientoEmpresas.log`, relativo al directorio de arranque. La carpeta se cambia con `logging.file.path`.

Comprobar el login de desarrollo:

```
curl -s -X POST http://localhost:8080/api/v1/auth/sso-login \
-H 'Content-Type: application/json' \
-d '{"dui":"00000000-0"}'
```

La respuesta trae el JWT en `data`. Sin el seed, ese DUI no existe.

`JWT_SECRET` no es una variable de entorno. Es la fila `JWT_SECRET` de `system_parameters`. La migración V15 deja una llave por defecto; cada ambiente puede reemplazarla. Cambiarla invalida las sesiones ya emitidas. El valor está en el manual de usuario, sección 10.1.

1.3 Frontend

```
cd CXP_DAV_CLIENT_REFACTOR
cp .env.example .env.development
npm install
npm run dev
```

Abrir `http://localhost:5173/financiamientocuentasporpagar/` .

Variable	Ejemplo local	Uso
VITE_API_BASE_URL	<code>http://localhost:8080</code>	Origen de la API, sin <code>/api</code> y sin barra final
VITE_PORTAL_URL	<code>https://devpay.davivienda.com.sv</code>	A dónde va el navegador sin sesión o al salir

Si falta una variable, la aplicación no arranca y muestra su nombre. Vite las lee al compilar: un cambio exige reiniciar `npm run dev` o volver a ejecutar `npm run build` .

El cliente arma la base de axios como `{VITE_API_BASE_URL}/api` . Con el ejemplo de arriba, el login llama a `http://localhost:8080/api/v1/auth/sso-login` .

El origen del front tiene que estar en el parámetro `CORS_ALLOWED_ORIGINS` . El valor inicial incluye `http://localhost:5173` . Se cambia en Gestión de parámetros (rol `SYSTEM_ADMIN`) y aplica sin reiniciar la API.

2. Cómo se hablan

```
Navegador -- /financiamientocuentasporpagar/ --> Vite o el servidor estático
Navegador -- Bearer JWT, /api/v1/... -----> API :8080
API        -- JDBC -----> PostgreSQL 16
```

- La sesión es stateless. El único endpoint público de negocio es `POST /api/v1/auth/sso-login` . Recibe `{ "dui": "..."}` y devuelve el JWT.
- El front guarda el token en `localStorage` con la clave `jwt_token` y lo envía como `Authorization: Bearer <token>` .
- `GET /api/v1/auth/me` devuelve el perfil, las rutas y los menús del rol. El front resuelve cada `componentName` en `COMPONENT_REGISTRY` de `src/App.jsx` .
- Un 401 con token presente cierra la sesión y redirige a `VITE_PORTAL_URL` .

- El cuerpo de las respuestas JSON es `ApiResponse` : `success` , `message` , `data` , `timestamp` . Los servicios del front leen `response.data.data` .
- Los roles son `ADMIN` , `PAYER` , `SUPPLIER` y `SYSTEM_ADMIN` . Lo que no está declarado en `SecurityConfig` responde 403. Que un pagador o un proveedor solo vea su entidad se valida en `CurrentUserService` .
- CORS permite `GET` , `POST` , `PUT` , `PATCH` , `DELETE` y `OPTIONS` , con los encabezados `Authorization` , `Content-Type` y `Accept` , y con credenciales.

3. Estructura

API, paquete `com.davivienda.factoraje` :

Paquete	Contenido
<code>controller</code>	REST bajo <code>/api/v1/...</code>
<code>service / service.impl</code>	Casos de uso
<code>domain</code>	Entidades, enums y constantes de rol
<code>repository</code>	Spring Data JPA
<code>dto</code>	Contratos de entrada y salida
<code>infrastructure.security</code>	JWT y el filtro de autenticación
<code>infrastructure.config</code>	Seguridad, parámetros y el pool de correo
<code>infrastructure.mail</code>	Avisos por Mailjet, después del commit

Frontend:

Carpeta	Contenido
<code>src/pages/<rol>/</code>	Pantallas de admin, operator, payer, supplier y system
<code>src/services/</code>	Llamadas a la API
<code>src/context/</code>	Sesión (<code>AuthContext</code>) y convenio seleccionado
<code>src/components/</code>	Piezas compartidas
<code>src/constants/</code>	URL, roles y rutas

La zona horaria de negocio es `America/El_Salvador` (`app.business-zone`).

4. Otro ambiente de desarrollo, sin Flyway

En `application.properties` hay dos líneas comentadas. Descomentarlas hace que Hibernate cree y actualice las tablas y que Flyway no corra. Van después de la configuración activa, así que la reemplazan. No usarlas en QA ni en producción.

```
spring.jpa.hibernate.ddl-auto=update
spring.flyway.enabled=false
```

Eso deja las tablas vacías: no inserta catálogos, rutas, menús ni parámetros. Esos datos están en `db/prod/inserts_iniciales.sql`.

5. QA y producción

QA usa el mismo camino que desarrollo: base vacía y Flyway al arrancar. No se carga `db/dev/seed_dev.sql`.

Producción no usa Flyway. La API arranca con `FLYWAY_ENABLED=false`. El DBA ejecuta una sola vez `db/prod/instalacion_inicial.sql` sobre una base PostgreSQL 16 vacía (esquema, catálogos, entidad banco y usuarios madre). Los cambios posteriores van en `db/prod/actualizaciones/`, en orden. El detalle, los parámetros que hay que completar y el motivo de no encender Flyway contra esa base están en `CXP_DAV_SERVER_REFACTOR/db/README.md`.

El empaquetado es `war`. `ServletInitializer` permite desplegarlo en un Tomcat externo. En local se arranca con la clase `FactorajeApplication`, que levanta el Tomcat embebido en el puerto 8080.

El front de un ambiente se genera con las variables de `.env.production`:

```
cd CXP_DAV_CLIENT_REFACTOR
npm install
npm run build
```

`dist/` se publica de forma que la aplicación quede bajo `/financiamientocuentasporpagar/` (base de `vite.config.js` y `basename` del router). `VITE_API_BASE_URL` debe ser el origen público de la API, y ese mismo origen del front debe estar en `CORS_ALLOWED_ORIGINS`.

Antes de dar por cerrado un ambiente, revisar en `system_parameters`:

- `CORS_ALLOWED_ORIGINS`, con la URL real del front.
- `JWT_SECRET`, si no debe usarse la llave por defecto.

- `MAILJET_API_KEY` , `MAILJET_API_SECRET` , `MAILJET_FROM_EMAIL` , `MAILJET_FROM_NAME` y `APP_LOGIN_URL` . Nacen en `CONFIGURAR` . Mientras sigan así, el correo no se envía y el resto de la operación sigue. `MAILJET_API_URL` ya apunta a `https://api.mailjet.com/v3/send` .

6. Pruebas

Desde `CXP_DAV_SERVER_REFACTOR` :

```
mvn test
```

`InstalacionInicialScriptTest` necesita Docker: compara `db/prod/instalacion_inicial.sql` con las migraciones de Flyway.

Desde `CXP_DAV_CLIENT_REFACTOR` :

```
npm test  
npm run lint
```